

Image Watermarking Tool

A Brief: what it does, how it is built, and where it leaks

Portfolio explainer

1 The one-paragraph version

A tkinter desktop app, 389 lines in a single `main.py`, that stamps a logo onto a batch of photos. You load one watermark image and any number of photos, use sliders to set the logo's size, opacity and position, watch a live before/after preview update as you drag, and then save every photo in one click. Originals are never touched; output goes to a `watermarked/` subfolder. One external dependency (Pillow, for pixels); tkinter ships with Python and supplies the window.

The idea worth remembering

The interesting design decision is a strict split between **four pure functions** that manipulate pixels and **one class** that manages the window. The live preview and the final save are two separate pipelines, but both funnel through the *same* compositing function. That is what makes the preview trustworthy rather than an approximation. This document explains where that promise holds and the one place it does not.

2 Architecture at a glance

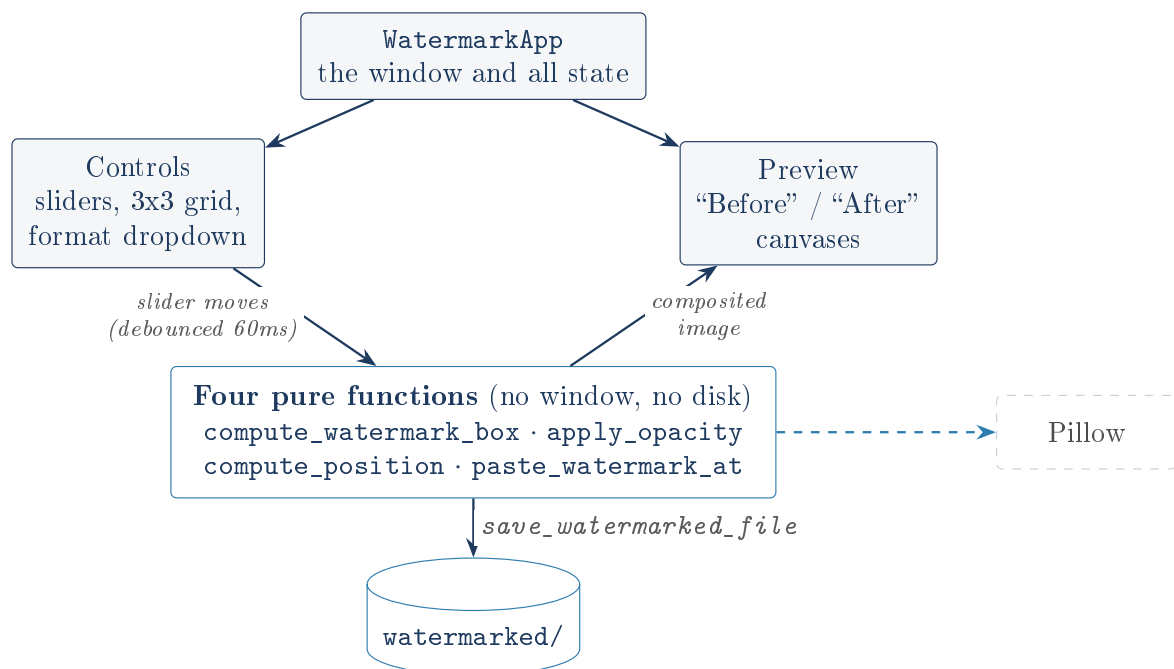


Figure 1: Everything above the blue box is interface. The blue box is where pixels actually move, and it is callable from either direction.

2.1 What each pure function does

<code>compute_watermark_box</code>	shrinks a <i>copy</i> of the logo to fit a box, aspect ratio kept
<code>apply_opacity</code>	scales the logo's alpha channel by a percentage
<code>compute_position</code>	preset (one of nine) plus a percentage-of-photo nudge
<code>paste_watermark_at</code>	blends the logo onto the photo

Two details carry more weight than their size suggests. The `copy()` in the first function is what makes the sliders non-destructive: the uploaded watermark is stored once and treated as read-only forever, and every render rebuilds a throwaway working copy from it. And the offsets in the third are percentages *of the photo*, not pixels, which is why one set of settings places the logo sensibly across a batch of mixed resolutions.

3 The war story: `paste()` does not blend

This is the project's centrepiece and the thing to be able to tell.

Pillow's `paste(image, (x, y))` with two arguments does not alpha-composite. It *overwrites* the destination pixels, copying the watermark's colour and alpha straight over the photo, which is discarded. The trap is that the output still looks plausible: the faded alpha value does land in the saved file, so any viewer renders the watermark as translucent. It is translucent against nothing.

The fix is a third argument, `paste(watermark, (x, y), watermark)`, which passes the watermark as its own mask and tells Pillow to treat the alpha channel as a blending recipe rather than as data to copy.

Path	Pixel under the logo	
<code>paste(wm, (x,y))</code>	(255, 0, 0, 127)	blue is <i>zero</i> : no blending happened
<code>paste(wm, (x,y), wm)</code>	(127, 0, 128, 191)	a real mix of both colours

The experiment: a solid blue photo, a solid red logo faded to 50%, the same pixel read under both paths. The argument is airtight, which is why it is worth stating this way: the base was pure blue, so if any blending had occurred, the blue channel could not possibly read zero. Both numbers were reproduced independently for this document rather than taken from the README on trust.

Why this is the strongest thing in the project

The broken version looks correct in almost any image viewer. Eyeballing the output would never have caught it. It was caught by constructing a synthetic case where the correct answer was known in advance and reading the actual pixel. That instinct, and not the fix itself, is the transferable skill here.

4 Preview and save: one promise, one leak

The README claims what is shown on screen is what gets written to disk. Mostly true, and the design is sound.

The preview thumbnails the first photo to a 300px box, scales the requested watermark size and offsets by the same ratio, composites, and draws. The save loops every photo at full resolution and rebuilds the watermark for that photo's real dimensions. Both then call the identical `apply_opacity` → `compute_position` → `paste_watermark_at` chain, so opacity and placement genuinely are WYSIWYG. Placement is safe by construction, since the offsets are percentages and the preview's base is a proportional thumbnail.

The leak: thumbnail shrinks but never enlarges

`Image.thumbnail` only ever makes things smaller. The preview asks for a box of `slider * scale`; the save asks for `slider`. When the slider value sits *above* the logo's native size, those two requests behave differently: the preview's smaller request still shrinks the logo, while the save's larger request does nothing at all.

Measured on a 3000x2000 photo with a 200x100 logo and the slider at 600: the preview renders the logo at **20%** of the canvas width, the saved file at **6.7%**. Three times too big on screen. With the same logo at slider 150, or with a logo natively larger than the slider, both paths agree exactly.

The bug gets easier to hit the larger the source photo, because a bigger photo means a smaller `scale`, pushing the preview's request further down. The root cause is an asymmetric Pillow operation used as though it were symmetric. The same fact also means the size sliders silently do nothing across their whole upper range for any logo smaller than the slider value.

5 Honest findings

The JPEG option in the format dropdown is unreachable

`apply_opacity` unconditionally converts to RGBA, so the composited result is always RGBA, so `save_watermarked_file`'s `mode == 'RGBA'` test is always true, so the output is always PNG. Verified: a save with the format set to 'JPEG' produced a .png file. The dropdown offers a choice, defaults to JPEG, and has no effect on any possible run.

The README flags this as "RGBA output still forces PNG regardless of the user's format choice", which is honest but frames it as an edge case. Tracing the callers shows it is unconditional: the control is not occasionally overridden, it is inert. The fix is either to delete it or to flatten onto a white background when JPEG is requested, which is what it implicitly promises.

Other real defects, in descending order of importance

- **No tests, at all.** Every claim rests on ad hoc scripts pasted into the README rather than anything re-runnable. The four pure functions are trivially testable, and both bugs above would have been caught by a three-line assertion each. This is the clearest gap.
- **Nothing clamps position to the canvas.** A 3000x2000 photo, bottom-right preset, both offsets at +20 yields (3400, 2300): entirely off the photo. Pillow clips silently, the logo vanishes, and the app still reports success. The preview shows the same emptiness, which is the saving grace.
- **Failed photos are skipped silently.** A photo that will not open is caught by a bare `except Exception` and continued past. The count stays accurate, but the user is told "Saved 9" after selecting ten and never learns which one failed.
- **The status label is always red,** including on success, because `fg='#b00020'` is set once and never changed.
- **Dead code.** `import os` is never used. Both `else` branches in `paste_watermark_at` are unreachable, since the watermark is always RGBA by the time it arrives.
- **The README's "Challenges" section** includes a LaTeX/pgfplots problem from building these explainer PDFs. Real, but it is a documentation-toolchain issue sitting in a list otherwise about alpha compositing, and it reads as padding.

Reopening the photo on every render

`_render_preview` calls `Image.open` on the base photo every single tick, with no cache. The 60ms debounce (which collapses a burst of slider events into one render) is the only thing keeping this survivable. On a very large source photo a slider drag becomes a sustained re-decode loop. The README flags this honestly under “What I’d do differently”; the fix is roughly three lines.

6 What is genuinely good

- The pure-function split is real, and it is why the compositing is trustworthy from two call sites. Most tools this size smear pixel logic through the button handlers.
- The read-only `watermark_original` gives non-destructive controls with no undo machinery.
- The alpha bug was proved, not eyeballed, against a case whose answer was known in advance.
- Percentage offsets are the right abstraction for a batch tool.
- Debounce, thumbnailed preview and shared compositing show the author trading responsiveness against correctness deliberately rather than picking one.

7 Glossary

tkinter

Python’s bundled GUI toolkit: windows, buttons, sliders. Knows nothing about pixels.

Pillow (PIL)

The image library: reading, resizing, fading, pasting, writing. Knows nothing about windows. The two meet only at `ImageTk.PhotoImage`.

Alpha channel / RGBA

A fourth number per pixel (0 to 255) recording opacity, alongside Red, Green and Blue. JPEG has none; PNG can.

Alpha compositing

Computing the correct blended colour when a partly transparent image lies over another. The thing `paste()` does not do by default.

Pure function

Output depends only on its inputs; changes nothing outside itself. Safe to call from two places and trust to behave identically.

Bounding box

`thumbnail((w,h))` shrinks an image to *fit inside* `w` by `h` while keeping its shape, rather than resizing to exactly that. Hence sliders labelled “Max”.

Aspect ratio

Width divided by height. Preserving it means never stretching.

Debouncing

On each of a rapid burst of events, cancel the previously scheduled work and reschedule it. A drag’s worth of events collapses into one render, 60ms after the last movement.

Event loop

The wait-for-input loop a GUI lives in, started by `mainloop()`. `after(ms, fn)` schedules work on it; not a thread, so no locks are needed.

Tk variables (`IntVar`, `StringVar`)

Wrapper objects that can notify widgets when their value changes. Read with `.get()`, written with `.set()`.

Geometry manager

`pack` (stack along an edge) or `grid` (rows and columns). Mixing both in one container is an error, which this code avoids by giving the offset sliders their own sub-frame.

`pathlib.Path`

Modern path handling. `.parent`, `.stem`, and `/` to join, with the right separator per operating system.

WYSIWYG

What You See Is What You Get: the on-screen view matches the final output exactly.

Verification status

The pixel claims here were re-derived by running the project's own functions under Python 3.12.3 and Pillow 10.1.0: the alpha table, the RGBA mode tracing, the non-enlargement of `thumbnail`, the preview/save divergence percentages, the off-canvas coordinates, and the JPEG-request-yields-PNG result.

Not verified, and therefore **inferred** from reading the source: the interactive flow through the native file dialogs (they block on user input and cannot be driven headlessly), and all behaviour on Windows and macOS.